Guía de estudio - Curso de Python (Santander Open Academy)

Cada sección indica el tema del curso, las ideas clave y el archivo `.py` donde está el código para practicar.

---

1. Introducción a Python

1.1 Instalación y configuración - Python es un lenguaje interpretado, de sintaxis sencilla y de propósito general. - Se instala desde python.org; para practicar se puede usar Jupyter Notebook (`pip install jupyter` , se abre con `jupyter notebook`). - `pip` es el gestor de paquetes.

1.2 Tu primer programa → `1.2 Tu primer programa en Python.py` - `print("¡Hola, Mundo!")` muestra texto en pantalla. - Python usa **sangría (indentación)** para definir bloques, no llaves. - Comentarios: `#` (una línea) y `""" ... """` (varias líneas). - Varias instrucciones en una línea se separan con `;` . - Los paréntesis cambian el orden de las operaciones: `(a + b) * c` .

2. Fundamentos

2.1 Variables → `2.1 Variables.py` - Una variable es una etiqueta que guarda un valor; se crea con `=` . - No se declara el tipo: Python lo infiere (`str` , `int` , `float` , `bool`). - Asignación múltiple: `a = b = c = 10` . - Nombres válidos: letras, números y `_` ; **no** pueden empezar con número, llevar guiones ni ser palabras reservadas (`if` , `for` ...). - `type(variable)` muestra el tipo.

2.2 Operadores → `2.2 Operadores.py` | Tipo | Operadores | |---|---| | Aritméticos | `+ - * / // % **` | | Comparación | `== != > < >= <=` | | Lógicos | `and or not` | - `/` da decimal, `//` división entera, `%` residuo, `**` potencia. - `=` asigna, `==` compara (error muy común).

3. Estructuras de control

Condicionales → `3. Estructuras de control.py` - `if` , `elif` , `else` : se evalúan en orden y se ejecuta solo el primer bloque verdadero. - Después de la condición van `:` y el bloque sangrado.

3.1 Bucles → `3.1 Bucles-loops.py` - `for x in secuencia:` recorre elementos; `range(n)` genera 0..n-1. - `while condicion:` repite mientras sea verdadera (cuidado con bucles infinitos). - `break` sale del bucle, `continue` salta a la siguiente vuelta, `pass` no hace nada.

4. Estructuras de datos

Listas → `4. Estructuras de datos.py` - Ordenadas y **modificables**: `[1, "a", 3.5]` . - Índices desde 0; negativos cuentan desde el final (`-1` = último). - Métodos: `append` , `insert` , `remove` , `pop` , `sort` , `reverse` . - Comprensión de listas: `[expresion for x in secuencia if condicion]` .

4.1 Tuplas → `4.1 Tuplas.py` - Ordenadas e **inmutables**: `(3, 4)` . - Método `index(valor, inicio, fin)` devuelve la posición.

4.2 Diccionarios → `4.2 Diccionarios.py` - Pares `clave: valor` : `{"nombre": "Juan"}` ; se accede por clave. - Métodos: `keys()` , `values()` , `items()` , `update()` .

4.3 Conjuntos (set) → `4.3 Conjuntos (set).py` - No ordenados y **sin duplicados**: `{1, 2, 3}` . - Operaciones: unión `|` , intersección `&` , diferencia `-` , diferencia simétrica `^` . - Métodos: `add` , `remove` (da error si no existe), `discard` (no da error), `clear` .

Estructura	Ordenada	Modificable	Duplicados
Lista	Sí	Sí	Sí
Tupla	Sí	No	Sí
Diccionario	Sí (por inserción)	Sí	Claves únicas
Set	No	Sí	No

5. Funciones → `5. Funciones.py`

- Se definen con `def nombre(parametros):` y se llaman con `nombre(argumentos)` .
- `return` devuelve un valor; sin él la función devuelve `None` .
- `lambda x: x ** 2` crea una función anónima de una línea.
- **Alcance:** variable local (solo dentro de la función) vs. global (todo el programa).
- **Docstring:** texto entre `""" """` justo debajo del `def` que documenta la función.
- `*args` permite recibir un número variable de argumentos.

6. Manejo de errores y excepciones

Tipos de error → `6. Manejo de errores y excepciones.py` - `SyntaxError` : sintaxis mal escrita (falta `:`). - `NameError` : variable no definida. - `TypeError` : tipos incompatibles (`5 + "10"`). - `IndexError` : índice fuera de rango.

6.1 Manejo de excepciones → `6.1 Manejo de excepciones.py` - `try` : código que puede fallar. `except Tipo` : qué hacer si falla. `finally` : se ejecuta siempre. - Se pueden encadenar varios `except` para distintos errores.

6.2 Excepciones personalizadas → `6.2 Excepciones personalizadas.py` - `raise Exception("mensaje")` lanza un error propio. - `except Exception as e:` captura el error y permite mostrar `e` .

7. Entradas y salidas

Entrada/salida → `7. Entradas-salidas.py` - `input("texto")` pide datos y **siempre devuelve texto**; usar `int()` o `float()` para convertir. - f-strings: `f"Hola {nombre}"` insertan variables en el texto.

7.1 Archivos → `7.1 Lectura y escritura de archivos.py` - `open("archivo", modo)` : `"r"` leer, `"w"` escribir (sobrescribe). - `read()` lee todo; `write()` escribe; `close()` cierra. - `with open(...) as f:` cierra el archivo automáticamente (forma recomendada).

8. Módulos y paquetes

8. Importación → `8. Importación y creación de módulos.py` - `import math` / `from math import sqrt` . - Biblioteca estándar: `math` , `random` , `datetime` .

8.1 Módulos propios → carpeta `8.1 Creación de módulos propios/` - Un módulo es un archivo `.py` con funciones; se usa con `import nombre_archivo`.

8.2 Paquetes → carpeta `8.2 Paquetes/` - Un paquete es una carpeta con `__init__.py` y varios módulos. - Se importa con `from paquete import modulo`.

Cómo estudiar

1. Lee el tema, ejecuta el archivo `.py` y **cambia los valores** para ver qué pasa.
2. Haz el `Ejercicio de ...` sin mirar la solución y luego compara.
3. Provoca errores a propósito y lee el mensaje que da Python.
4. Repasa las tablas de arriba (operadores y estructuras de datos) hasta recordarlas sin mirar.

Autopreguntas de repaso

- ¿Qué diferencia hay entre `=` y `==`?
- ¿Cuándo usar lista, tupla, diccionario o set?
- ¿Qué hace `finally` y cuándo se ejecuta?
- ¿Por qué `input()` necesita `int()` para comparar números?
- ¿Qué diferencia hay entre `remove` y `discard` en un set?
- ¿Qué convierte una carpeta en un paquete?